

CipherScope

Feature Documentation

CipherScope is a browser-based cryptography learning workbench for implementing, running, attacking, and comparing classical, symmetric-key, and public-key cryptographic algorithms.

No Database

Python Backend

Static Frontend

Manual Core Algorithms

CSE721 Project

1. Project Overview

CipherScope provides a complete educational interface for experimenting with cryptographic algorithms. The frontend collects text, keys, and parameters, while the backend performs all algorithm operations and

returns structured JSON results. The application is intentionally database-free and focuses on transparent algorithm behavior, security analysis, and performance measurement.

Primary Goal

Demonstrate encryption, decryption, key generation, cryptanalysis, and comparative evaluation in one browser-accessible workbench.

Technology Model

Static HTML, CSS, and JavaScript frontend served by a Python HTTP backend.

Algorithm Scope

Substitution, Caesar attack, double transposition, DES, AES-128, RSA, ECC, and ECDH.

Learning Focus

Clear intermediate outputs such as frequency tables, round keys, generated keys, attack candidates, and timing measurements.

2. System Architecture

CipherScope uses a simple two-layer architecture. The backend serves the frontend files and exposes API endpoints under `/api/`. The frontend uses `fetch()` requests to send JSON payloads and displays backend responses directly in result panels or comparison tables.

- **Frontend:** `frontend/index.html`, `frontend/style.css`, and `frontend/script.js`.
- **Backend server:** `backend/app.py`, built with Python's standard `http.server`.
- **Classical algorithms:** substitution cipher, Caesar brute force, and double transposition.
- **Symmetric algorithms:** manually implemented DES and AES-128 demonstrations.
- **Public-key algorithms:** RSA, ECC point operations, and ECDH shared-key demonstration.
- **Analysis layer:** security comparison data and a live performance comparison table.

3. Major Features

Area	Feature	Inputs	Outputs
Classical	Substitution cipher encryption and decryption	Plaintext or ciphertext plus a 26-letter substitution key	Ciphertext, plaintext, and ciphertext frequency analysis
Classical	Substitution frequency analysis and heuristic attack	Text or substitution ciphertext	Letter counts, frequency order, guessed mapping, and suggested plaintext
Classical	Caesar brute-force attack demo	Caesar ciphertext	All shift candidates for manual inspection
Classical	Double transposition encryption and decryption	Text, row permutation key, and column permutation key	Ciphertext, intermediate transformation, padding note, and frequency analysis
Classical	Double transposition permutation attack	Double transposition ciphertext	Candidate decryptions for row and column sizes from 1 through 4
Symmetric	DES encryption and decryption	Plaintext for encryption or ciphertext hex and key hex for decryption	Generated key, ciphertext hex, 16 round keys, plaintext, and timing
Symmetric	AES-128 encryption and decryption	Plaintext for encryption or ciphertext hex and key hex for decryption	Generated key, ciphertext hex, 11 round keys, plaintext, and timing
Public-Key	RSA key generation, encryption, and decryption	Key size, plaintext, ciphertext, public exponent, modulus, and private exponent	p, q, n, phi, e, d, public key, private key, ciphertext, and recovered plaintext
Public-Key	RSA factorization attack demo	Modulus n and optional public exponent e	Recovered p and q for small demo moduli, plus recovered d when possible
Public-Key	ECC point listing and public-key generation	Curve parameters p, a, b, base point G, n, and private key	Valid finite curve points, selected private key, base point, and public key
Public-Key	ECDH shared key demonstration	Alice and Bob private keys plus curve parameters	Alice public key, Bob public key, both shared keys, and equality check
Analysis	Security and performance comparison	User-triggered comparison actions	Static security table and live technical timing table for all six algorithm families

4. Frontend Experience

The interface is organized into four major tabs: Classical, Symmetric, Public-Key, and Comparison. Each tab keeps the expected controls close to the related algorithm and uses mode selectors to reveal only the relevant input fields.

- **Classical tab:** substitution cipher, Caesar brute-force demo, frequency analysis, and double transposition.
- **Symmetric tab:** DES and AES-128 encryption/decryption panels with generated key output.
- **Public-Key tab:** RSA workflows, ECC point listing, ECC public key generation, and ECDH.
- **Comparison tab:** security comparison and live performance measurement tables.

Result panels display structured JSON so students can inspect every important output instead of seeing only a final ciphertext or plaintext.

5. Backend API Summary

Endpoint	Purpose
GET /api/health	Confirms that the CipherScope backend is running.
GET /api/comparison	Returns the security comparison table.
GET /api/performance-comparison	Runs the live technical performance comparison.
POST /api/substitution/encrypt	Encrypts plaintext using a monoalphabetic substitution key.
POST /api/substitution/decrypt	Decrypts substitution ciphertext.
POST /api/substitution/frequency	Counts and ranks letter frequencies.
POST /api/substitution/frequency-attack	Builds a heuristic substitution attack guess.
POST /api/substitution/bruteforce	Runs the Caesar brute-force attack demo.
POST /api/double-transposition/encrypt	Encrypts text with row and column permutation keys.
POST /api/double-transposition/decrypt	Decrypts double transposition ciphertext.
POST /api/double-transposition/frequency	Runs frequency analysis on double transposition text.
POST /api/double-transposition/permutation-attack	Lists small-key permutation attack candidates.
POST /api/des/encrypt	Encrypts plaintext with a generated DES key.
POST /api/des/decrypt	Decrypts DES ciphertext hex using the supplied key.
POST /api/aes/encrypt	Encrypts plaintext with a generated AES-128 key.
POST /api/aes/decrypt	Decrypts AES ciphertext hex using the supplied key.
POST /api/rsa/generate-keys	Generates RSA p, q, n, phi, e, d, public key, and private key.
POST /api/rsa/encrypt	Encrypts plaintext with RSA public-key values.
POST /api/rsa/decrypt	Decrypts RSA ciphertext with private-key values.

Endpoint	Purpose
POST /api/rsa/factorization-attack	Attempts trial-division factorization for demo-sized RSA moduli.
POST /api/ecc/points	Lists finite points on a small elliptic curve.
POST /api/ecc/generate-key	Generates an ECC public key from a private scalar.
POST /api/ecc/ecdh	Runs an ECDH shared-key exchange demonstration.

6. Algorithm Notes

Substitution Cipher

The substitution module cleans text to uppercase letters and spaces, validates that the key contains all 26 letters exactly once, and preserves spaces during encryption and decryption. The frequency attack maps ranked ciphertext letters to common English frequencies as a heuristic learning aid.

Double Transposition

Double transposition separates letters from spaces, applies a row permutation followed by a column permutation, and restores spaces into the output template. Padding uses **X** when a block is incomplete. The attack mode enumerates possible row and column permutations for small dimensions.

DES

DES is implemented manually with initial/final permutations, expansion, S-boxes, P-box permutation, 16 Feistel rounds, generated 8-byte keys, PKCS-style padding, and displayed round keys.

AES-128

AES-128 is implemented manually with SubBytes, ShiftRows, MixColumns, AddRoundKey, inverse operations, key expansion, generated 16-byte keys, padding, and displayed round keys.

RSA

RSA includes prime generation, Miller-Rabin style probabilistic primality testing, modular inverse calculation, key generation, direct integer encryption/decryption, and a trial-division factorization demo for small keys.

ECC and ECDH

ECC supports curve validation, point enumeration, point addition, scalar multiplication, public-key generation, and an ECDH exchange that verifies Alice and Bob derive the same shared point.

7. Security and Educational Scope

CipherScope is designed for learning and demonstration, not for protecting real secrets. The project intentionally exposes intermediate values and uses simplified educational flows so students can observe how each algorithm behaves.

- DES is historically important but no longer recommended for modern security.
- Raw RSA without production padding such as OAEP is not secure for real applications.
- The RSA factorization attack is limited by a fixed demo step count and is expected to fail on large keys.
- The ECC module uses small curve examples for browser-friendly learning, not production-grade curves.
- Generated keys use Python's general random module for demonstration rather than a hardened production key-management system.

- The application has no authentication, persistence, access control, or audit logging because it is a local educational project.

8. Running and Testing

Start the application from the project directory:

```
python3 backend/app.py
```

Then open:

```
http://localhost:8000
```

Run the algorithm tests with:

```
python3 tests/test_algorithms.py
```

9. Suggested Future Enhancements

- Add input examples that can be loaded into each algorithm panel.
- Add CSV or JSON export for comparison results.
- Add graphical charts for timing and security comparison.
- Add stronger error presentation in the frontend instead of raw JSON only.
- Add optional step-by-step visualizations for DES rounds, AES rounds, RSA math, and ECC point multiplication.